Test Cases

Eurotoldo Event Management System — EventController & Inventory Modules

Project: Eurotoldo Event Management System	**Environment:** Apache NetBeans / JUnit 5.10	**Execution Date:** 2026-10-27
Modules Tested: EventController & Inventory	**Target Version / Build:** v1.0.0-final	**Execution Outcome:** 20/20 Passed (100.0%)

1. Test Specification Matrix — EventController Module

Test ID	Preconditions	Input	Expected Result	Actual Result	Status	Observations
EV-01	Event list initially empty	id: 1, name: 'Royal Wedding', location: 'Quito Tenis', startDate: 2026-06-10, endDate: 2026-06-12	addEvent() returns true and adds the event to the list.	Method returned true. Event registered in memory.	**Passed**	Valid multi-day event creation test.
EV-02	Event list initialized	id: 2, name: 'Tech Conference', location: 'ESPE Auditorium', startDate: 2026-07-15, endDate: 2026-07-15	addEvent() returns true for single-day events.	Method returned true. Event stored successfully.	**Passed**	Accepts events where start date equals end date.
EV-03	Event with ID 3 ('Corporate Party') pre-registered	Search ID: 3	findEventById() returns matching non-null Event object.	Non-null Event object returned matching name 'Corporate Party'.	**Passed**	Successful lookup by unique primary key ID.
EV-04	Event with ID 4 registered in list	id: 4, name: 'ESPE Graduation', location: 'Main Hall', startDate: 2026-09-01, endDate: 2026-09-03	updateEvent() returns true and updates fields for ID 4.	Method returned true. Name updated to 'ESPE Graduation'.	**Passed**	Preserves existing ID and updates all fields correctly.
EV-05	Event list contains 1 registered event (ID 5)	Call getEvents()	getEvents() returns list containing 1 event.	Returned List<Event> containing 1 item.	**Passed**	Verifies that returned collection size reflects registered events count.
EV-06	Event ID 6 updated location to 'Exposition Center'	Search ID: 6 after update	findEventById() returns event reflecting updated location.	Retrieved event location equals 'Exposition Center'.	**Passed**	Confirms in-memory persistence following updates.
EV-07	Controller instantiated	event = null	addEvent() returns false and does not add event.	Method returned false. Event list size remained 0.	**Passed**	Null object guard verified using assertFalse().
EV-08	Event with ID 8 ('Concert 1') already exists	id: 8, name: 'Concert 2', location: 'Coliseum', startDate: 2026-06-05, endDate: 2026-06-06	addEvent() returns false and rejects duplicate ID.	Method returned false. Duplicate event was not added.	**Passed**	Primary key uniqueness rule verified using assertFalse().
EV-09	Controller instantiated	id: 9, name: 'Private Meeting', location: ' ', startDate: 2026-07-01, endDate: 2026-07-02	addEvent() returns false and rejects whitespace location.	Method returned false. Invalid event rejected.	**Passed**	Blank string input validation verified using assertFalse().

Test ID	Preconditions	Input	Expected Result	Actual Result	Status	Observations
EV-10	Controller instantiated	id: 10, name: 'Book Fair', location: 'Metropolitan Park', startDate: 2026-08-10, endDate: 2026-08-05	addEvent() returns false for endDate prior to startDate.	Method returned false. Chronological error rejected.	**Passed**	Date range rule validation verified using assertFalse().

2. Test Specification Matrix — Inventory Module

Test ID	Preconditions	Input	Expected Result	Actual Result	Status	Observations
INV-01	Inventory equipment list initially empty	id: 1, name: 'Tent 10x10', totalQuantity: 15	addEquipment() returns true and adds equipment to inventory.	Method returned true. Inventory size equals 1.	**Passed**	Valid equipment addition with positive quantity.
INV-02	Inventory list initialized	Eq 1: (id: 1, qty: 5), Eq 2: (id: 2, qty: 100)	addEquipment() returns true for both independent additions.	Method returned true for both calls. Inventory size equals 2.	**Passed**	Sequential addition of multiple distinct equipment items.
INV-03	Equipment ID 10 ('Round Table') pre-registered	Search ID: 10	findEquipmentById() returns matching non-null Equipment object.	Non-null Equipment returned matching name 'Round Table'.	**Passed**	Successful primary key search in equipment collection.
INV-04	Inventory initialized	id: 3, name: 'LED Display', totalQuantity: 0	addEquipment() returns true (stock 0 allowed for catalog entry).	Method returned true. Item registered with stock 0.	**Passed**	Lower bound check for catalog quantity (zero stock accepted).
INV-05	Inventory contains 1 registered item ('Generator')	Call getEquipmentList()	getEquipmentList() returns collection containing registered item.	Returned list containing 1 item ('Generator').	**Passed**	Collection getter access verified.
INV-06	Items with ID 1 and ID 2 present in inventory list	Search ID: 2	findEquipmentById() returns Equipment object with ID 2.	Target Equipment object retrieved successfully.	**Passed**	Complete loop traversal search verified.
INV-07	Inventory initialized	equipment = null	addEquipment() returns false and rejects null input.	Method returned false. Inventory size remained unchanged.	**Passed**	Null pointer guard verified using assertFalse().
INV-08	Equipment ID 1 ('Blue Tent') already registered	id: 1, name: 'Red Tent', totalQuantity: 5	addEquipment() returns false and rejects duplicate ID.	Method returned false. Duplicate item not added.	**Passed**	Equipment ID uniqueness rule verified using assertFalse().
INV-09	Inventory initialized	id: 4, name: 'Projector', totalQuantity: -5	addEquipment() returns false and rejects negative quantity.	Method returned false. Item rejected.	**Passed**	Non-negative quantity rule verified using assertFalse().
INV-10	Inventory contains equipment items (IDs 1..5)	Search ID: 999	findEquipmentById() returns null for non-existent ID.	Method returned null as expected.	**Passed**	Search for missing item verified using assertNull().

4. Conclusions & JUnit Execution Verification

• **Correct Negative Test Assertions:** All negative scenarios now utilize proper JUnit assertions (assertFalse for invalid additions, assertNull for missing item searches) to verify that business logic correctly rejects invalid data (null objects, duplicate primary keys, empty/whitespace strings, inverted date ranges, and negative quantities). As a result, all 20 test cases pass successfully (100% Pass Rate).

• **Separation of Test Intention and Result:** Expected Result reflects expected system behavior rather than internal test mechanics. Preconditions explicitly define system state prior to test execution.